

Abstract

This lab demonstrates the creation of histograms and their utility in image processing, extending the capabilities of the point processes implemented in HW1A. Previously, pixels could only be analyzed according to their individual attributes. Histograms change this by providing information about the entire image. With that context, we are able to use the same pixel transformations to accomplish more complicated tasks.

Introduction

The first section of this lab focused on the generation of histograms. I first created a function to aggregate the intensity values of one color channel from an RGB image. I then limited my data collection to a region indicated by a mask. For the second part of this lab, I implemented a Linear Contrast Stretch. This demonstrated the ways in which the information of a histogram can be helpful in processing images by altering the input image to use the entire intensity range.

Part B1: Histogram Computation



Original Image



Mask



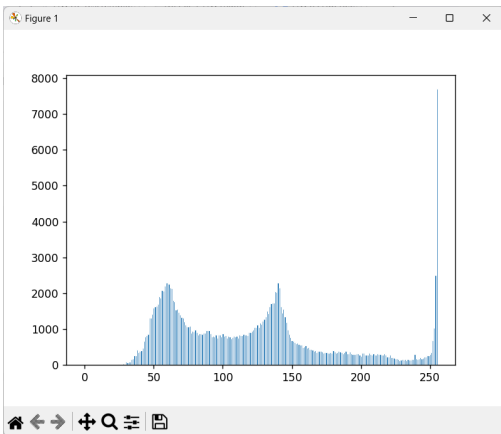
Red Channel



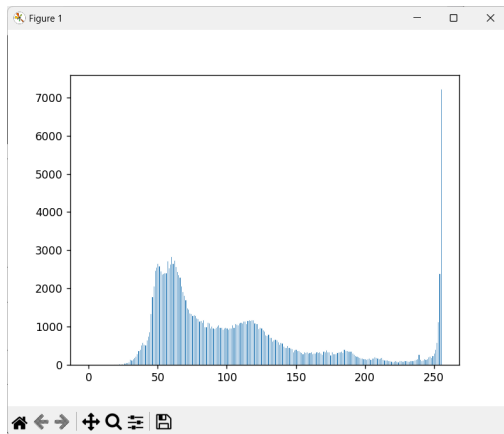
Green Channel



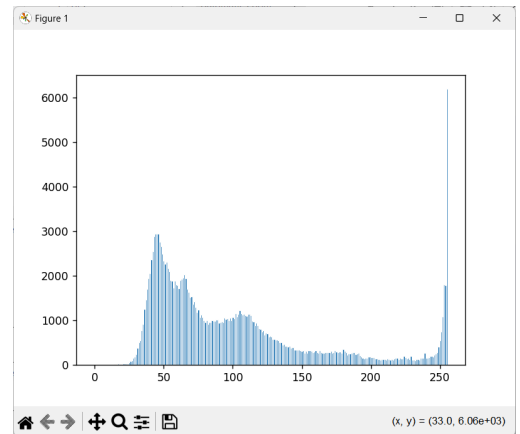
Blue Channel



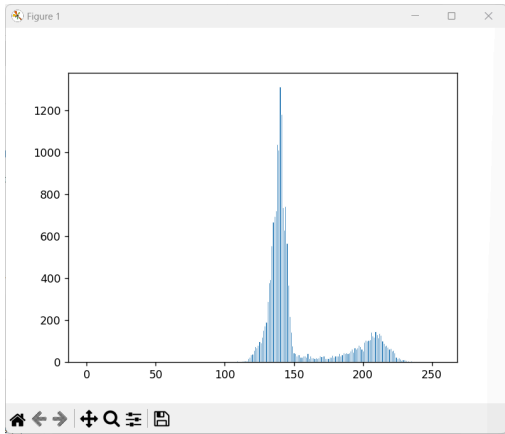
Red Histogram with 256 Bins



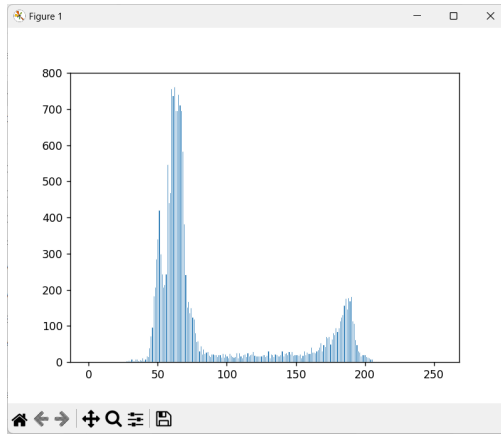
Green Histogram with 256 Bins



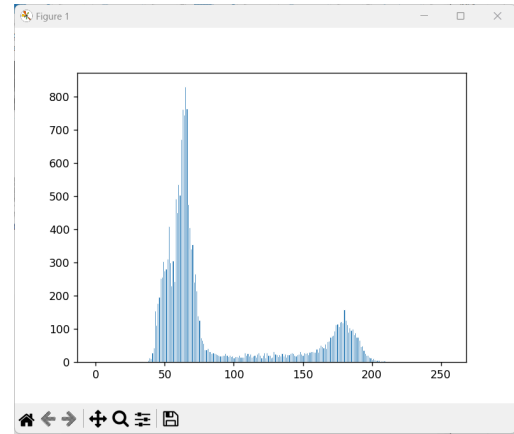
Blue Histogram with 256 Bins



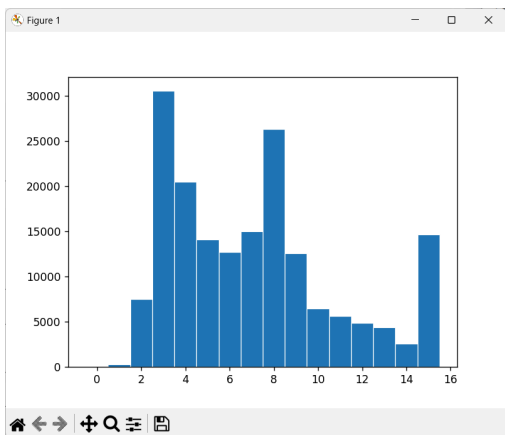
Masked Red Histogram with 256 Bins



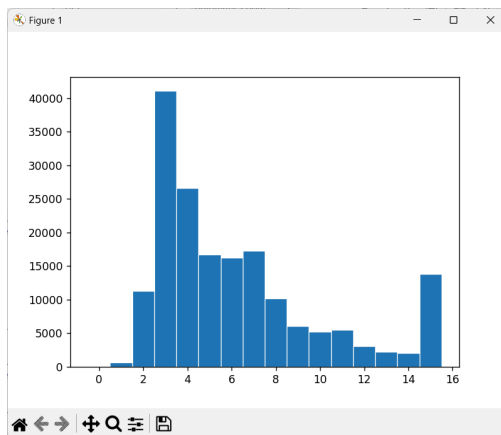
Masked Green Histogram with 256
Bins



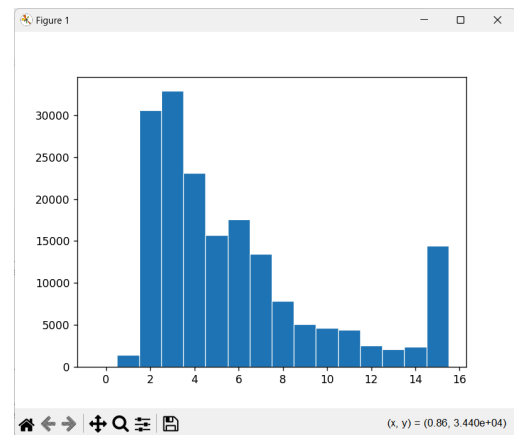
Masked Blue Histogram with 256 Bins



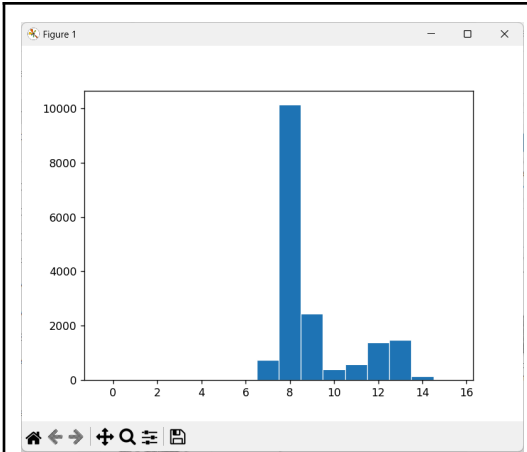
Red Histogram with 16 Bins



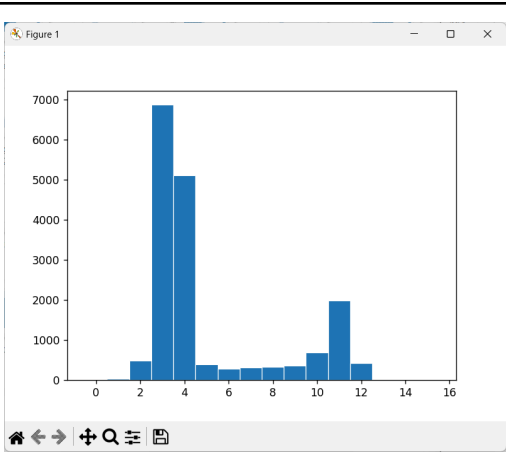
Green Histogram with 16 Bins



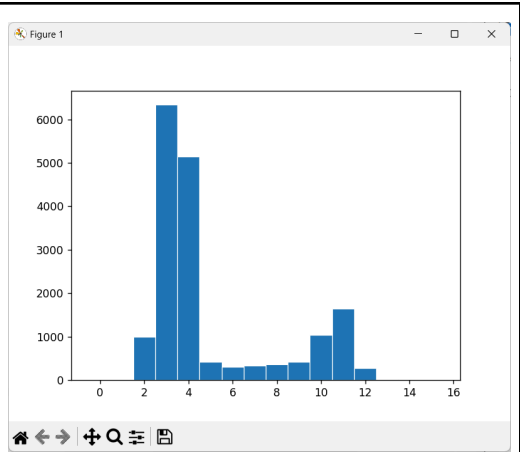
Blue Histogram with 16 Bins



Masked Red Histogram with 16 Bins



Masked Green Histogram with 16
Bins



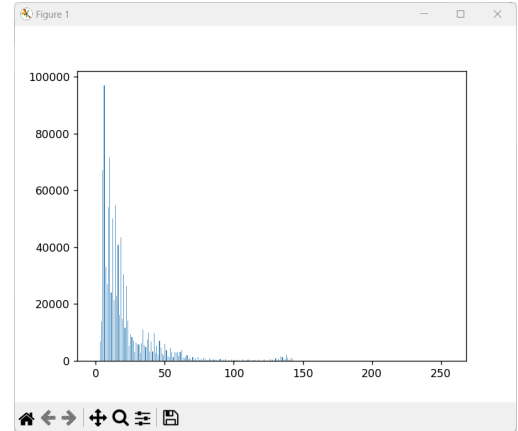
Masked Blue Histogram with 16 Bins

Processing Times	Red	Green	Blue
Original with 256 Bins	1.840s	1.769s	1.841s
Masked with 256 Bins	1.436s	1.481s	1.311s
Original with 16 Bins	1.914s	1.861s	1.886s
Masked with 16 Bins	1.475s	1.490s	1.447s

Part B2: Contrast Enhancement



Original Image



Original Histogram



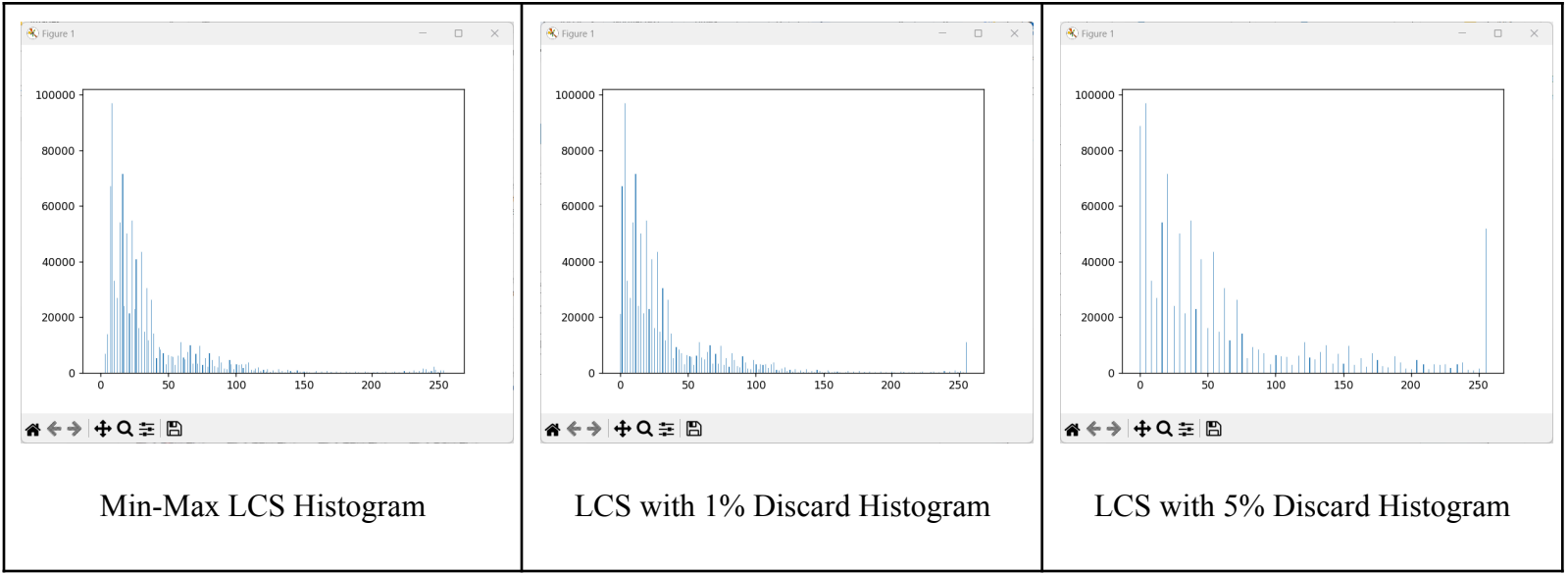
Min-Max LCS



LCS with 1% Discard



LCS with 5% Discard



	Input Minimum Intensity	Input Maximum Intensity	Output Minimum Intensity	Output Maximum Intensity
Original Images	1	143	1	143
Min-Max LCS	1	143	0	255
LCS with 1% Discard	4	134	0	255
LCS with 5% Discard	5	66	0	255

Discussion

Part 1

- Although I included the single channel grayscale images as requested by the test format, these were not produced by my histogram program. The process of generating these images would take more time than it would save by decreasing the image file size. As such, `rgb2histogram.py` accepts an RGB input image, the number of histogram bins, and the channel to create a histogram out of. It iterates through each pixel and only adds data from the specified channel to the resulting histogram.
- The histogram results are generally in line with what I expected. The full image histograms contain many more pixels than the masks, as indicated by the different y-scaling. As the RGB image is mostly muted tones, it makes sense for the three different channel histograms to look roughly the same. The only exception to this is the second peak in the Red histogram, which is a result of the large concentration of high intensity red pixels in the stop sign.
 - This second peak is essentially the area of the image expanded by the masked histogram. The vast majority of Green and Blue pixels are concentrated in the 50-80 intensity range, while the Red pixels are concentrated around the 125-150 range. All three histograms have a small peak near 200, which I assume is the white lettering of “STOP.”
- Reducing the number of bins mostly serves to smooth the overall trend of each channel’s intensity. All that can be meaningfully discerned is that Red has that second peak due to the stop sign, but otherwise the pixel concentration decreases as the intensity increases for each channel. There is one final peak at the end of each channel, which is due to the white parts of the image (the cars and the back of the other sign).
- I was initially confused as to why histogram creation was more efficient using the masked image considering that it is implemented by iterating through the original and the masked image simultaneously, then adding the

pixel value from the original image depending on the masked pixel's state. It seems that whatever extra cost this incurs is less than the cost of modifying a larger array, which the original histogram's data would involve.

- It does make sense that the 16 bin times are slightly longer than the 256 bin times, as translation for each of the number values is required. The 256 possible intensity values have to be assigned to 16 bins, so I saved time by performing these calculations once. Instead of computing the appropriate bin for each pixel's intensity, I created an array that holds all possible conversion values. Each pixel's intensity is checked against the corresponding element in this array to get the appropriate bin. This process is still more resource heavy than directly mapping the pixel intensity values, which using 256 bins allows us to do.

Part 2

- I performed the contrast stretch using the formula given in the lectures, where the Max-Min differential is multiplied with the desired range and then the pixel value. This is also true for the discard algorithms, but the minimum and maximum values are closer together allowing for a wider stretch.
- I managed pixel dropping by first calculating the number of pixels that make up the desired percent to drop. After generating the histogram, I ran a for loop iterating through the intensity values. I kept a running sum of the number of pixels in the already visited values, and checked whether that sum exceeded the desired drop amount (divided by two since the drop occurs on both sides). If so, that intensity is the new minimum. I repeated this for the maximum, which worked the same except the for loop was descending.
- Discarding 1% of pixels only decreased the Min-Max range by 12 points. Discarding 5% shot this number up to 81.
- The output minimum and maximum intensities were 0 and 255 for all contrast stretched images. I expected this for the drop algorithms, as the new intensity calculation for dropped pixels would put them outside of the

possible range, meaning they defaulted to either black or white. For the Min-Max intensity, the calculation works by converting the minimum to 0 and the maximum to 255, so it's not surprising that pixels with those values ended up filling the entire intensity range.

- All three stretched images are far brighter than the original, whose pixels were almost all below 75 intensity. The 1% discard image is not particularly different from the Min-Max LCS, but the 5% is dramatically brighter. More details are discernable in darker areas, but the brightest areas are completely washed out. Any pixels in the top 2.5% of intensity were defaulted to pure white, which obliterated detail in certain areas of the image. The bottom of the van illustrates this best, as the bumper is no longer clearly distinguished.